DUKE BIKE DATABASE MANAGEMENT SYSTEM

DATABASE RELATIONSHIP ANALYSIS USING JOINS

CHAPTER – 9

Objectives

1. Combine Customer, Product, Orders, Order_Details, and Payment tables.
2. Implement INNER JOIN, LEFT JOIN, and RIGHT JOIN.
3. Retrieve complete order details.
4. Display customer purchase history.
5. Generate multi-table reports.

Note: SQL is written for Oracle SQL*Plus. Join column names are based on a common Duke Bike table design; if your actual table columns differ, update the column names to match DESC output.

1. Combining Multiple Tables

The Duke Bike database stores customer, product, order, order-detail, and payment information. JOIN operations combine related records.

```
SELECT C.ID, C.NAME, O.ORDER_ID, O.ORDER_DATE, O.TOTAL_AMOUNT
FROM Customer C
INNER JOIN Orders O
ON C.ID = O.CUSTOMER_ID;
```

OUTPUT FORMAT (example)

```
SQL> SELECT C.ID, C.NAME, O.ORDER_ID, O.ORDER_DATE, O.TOTAL_AMOUNT
2 FROM Customer C
3 INNER JOIN Orders O ON C.ID = O.CUSTOMER_ID;
ID NAME ORDER_ID ORDER_DATE TOTAL_AMOUNT
---------- -------------------- -------- ---------- ------------
101 Customer A 5001 21-SEP-26 190000.00
102 Customer B 5002 21-SEP-26 12500.00
103 Customer C 5003 20-SEP-26 85000.00
104 Customer D 5004 20-SEP-26 4200.00
4 rows selected.
```

2. INNER JOIN – Orders and Payment

INNER JOIN displays only orders that have a matching payment record.

```
SELECT O.ORDER_ID, O.CUSTOMER_ID, O.TOTAL_AMOUNT,
P.PAYMENT_ID, P.PAYMENT_METHOD, P.PAYMENT_STATUS
FROM Orders O
INNER JOIN Payment P
ON O.ORDER_ID = P.ORDER_ID;
```

OUTPUT FORMAT (example)

```
SQL> SELECT O.ORDER_ID, O.CUSTOMER_ID, O.TOTAL_AMOUNT,
2 P.PAYMENT_ID, P.PAYMENT_METHOD, P.PAYMENT_STATUS
3 FROM Orders O INNER JOIN Payment P ON O.ORDER_ID = P.ORDER_ID;
ORDER_ID CUSTOMER_ID TOTAL_AMOUNT PAYMENT_ID PAYMENT_METHOD PAYMENT_STATUS
---------- ----------- ------------ ---------- -------------- --------------
5001 101 190000.00 9001 UPI PAID
5002 102 12500.00 9002 CARD PAID
2 rows selected.
```

2.1 INNER JOIN – Product and Order_Details

This query shows products included in each order.

```
SELECT OD.ORDER_ID, P.PRODUCT_ID, P.PRODUCT_NAME,
OD.QUANTITY, OD.SUBTOTAL
FROM Product P
INNER JOIN Order_Details OD
ON P.PRODUCT_ID = OD.PRODUCT_ID;
```

OUTPUT FORMAT (example)

```
SQL> SELECT OD.ORDER_ID, P.PRODUCT_ID, P.PRODUCT_NAME,
2 OD.QUANTITY, OD.SUBTOTAL
3 FROM Product P INNER JOIN Order_Details OD
4 ON P.PRODUCT_ID = OD.PRODUCT_ID;
ORDER_ID PRODUCT_ID PRODUCT_NAME QUANTITY SUBTOTAL
---------- ---------- ----------------- -------- --------
5001 201 Duke Road Bike 1 190000.00
5002 202 Bike Helmet 1 12500.00
2 rows selected.
```

3. LEFT JOIN – All Customers and Their Orders

LEFT JOIN returns every customer, including customers who do not yet have an order.

```
SELECT C.ID, C.NAME, O.ORDER_ID, O.ORDER_DATE, O.TOTAL_AMOUNT
FROM Customer C
LEFT JOIN Orders O
ON C.ID = O.CUSTOMER_ID
ORDER BY C.ID;
```

OUTPUT FORMAT (example)

```
SQL> SELECT C.ID, C.NAME, O.ORDER_ID, O.ORDER_DATE, O.TOTAL_AMOUNT
2 FROM Customer C LEFT JOIN Orders O ON C.ID = O.CUSTOMER_ID
3 ORDER BY C.ID;
ID NAME ORDER_ID ORDER_DATE TOTAL_AMOUNT
---------- -------------------- -------- ---------- ------------
101 Customer A 5001 21-SEP-26 190000.00
102 Customer B 5002 21-SEP-26 12500.00
105 Customer E (null) (null) (null)
3 rows selected.
```

4. RIGHT JOIN – All Orders and Customer Details

RIGHT JOIN returns every row from Orders and matching customer details when available.

```
SELECT C.ID, C.NAME, O.ORDER_ID, O.ORDER_DATE, O.TOTAL_AMOUNT
FROM Customer C
RIGHT JOIN Orders O
ON C.ID = O.CUSTOMER_ID
ORDER BY O.ORDER_ID;
```

OUTPUT FORMAT (example)

```
SQL> SELECT C.ID, C.NAME, O.ORDER_ID, O.ORDER_DATE, O.TOTAL_AMOUNT
2 FROM Customer C RIGHT JOIN Orders O ON C.ID = O.CUSTOMER_ID
3 ORDER BY O.ORDER_ID;
ID NAME ORDER_ID ORDER_DATE TOTAL_AMOUNT
---------- -------------------- -------- ---------- ------------
101 Customer A 5001 21-SEP-26 190000.00
102 Customer B 5002 21-SEP-26 12500.00
2 rows selected.
```

5. Complete Order Details

Combines Customer, Orders, Order_Details, Product, and Payment to show the full order record.

```
SELECT C.ID, C.NAME, O.ORDER_ID, O.ORDER_DATE,
P.PRODUCT_NAME, OD.QUANTITY, OD.SUBTOTAL,
O.TOTAL_AMOUNT, PY.PAYMENT_METHOD, PY.PAYMENT_STATUS
FROM Customer C
JOIN Orders O ON C.ID = O.CUSTOMER_ID
JOIN Order_Details OD ON O.ORDER_ID = OD.ORDER_ID
JOIN Product P ON OD.PRODUCT_ID = P.PRODUCT_ID
LEFT JOIN Payment PY ON O.ORDER_ID = PY.ORDER_ID
ORDER BY O.ORDER_ID;
```

OUTPUT FORMAT (example)

```
SQL> SELECT C.ID, C.NAME, O.ORDER_ID, O.ORDER_DATE,
2 P.PRODUCT_NAME, OD.QUANTITY, OD.SUBTOTAL,
3 O.TOTAL_AMOUNT, PY.PAYMENT_METHOD, PY.PAYMENT_STATUS
4 FROM Customer C JOIN Orders O ON C.ID = O.CUSTOMER_ID
5 JOIN Order_Details OD ON O.ORDER_ID = OD.ORDER_ID
6 JOIN Product P ON OD.PRODUCT_ID = P.PRODUCT_ID
7 LEFT JOIN Payment PY ON O.ORDER_ID = PY.ORDER_ID
8 ORDER BY O.ORDER_ID;
ID NAME ORDER_ID PRODUCT_NAME QTY SUBTOTAL TOTAL_AMOUNT PAYMENT
-- ---------- -------- --------------- --- --------- ------------ -------
101 Customer A 5001 Duke Road Bike 1 190000.00 190000.00 UPI
102 Customer B 5002 Bike Helmet 1 12500.00 12500.00 CARD
2 rows selected.
```

6. Customer Purchase History

Displays each customer's orders and the products purchased.

```
SELECT C.ID, C.NAME, O.ORDER_ID, O.ORDER_DATE,
P.PRODUCT_NAME, OD.QUANTITY, OD.SUBTOTAL
FROM Customer C
JOIN Orders O ON C.ID = O.CUSTOMER_ID
JOIN Order_Details OD ON O.ORDER_ID = OD.ORDER_ID
JOIN Product P ON OD.PRODUCT_ID = P.PRODUCT_ID
ORDER BY C.ID, O.ORDER_DATE;
```

OUTPUT FORMAT (example)

```
SQL> SELECT C.ID, C.NAME, O.ORDER_ID, O.ORDER_DATE,
2 P.PRODUCT_NAME, OD.QUANTITY, OD.SUBTOTAL
3 FROM Customer C JOIN Orders O ON C.ID = O.CUSTOMER_ID
4 JOIN Order_Details OD ON O.ORDER_ID = OD.ORDER_ID
5 JOIN Product P ON OD.PRODUCT_ID = P.PRODUCT_ID
6 ORDER BY C.ID, O.ORDER_DATE;
ID NAME ORDER_ID PRODUCT_NAME QUANTITY SUBTOTAL
-- ---------- -------- --------------- -------- ---------
101 Customer A 5001 Duke Road Bike 1 190000.00
102 Customer B 5002 Bike Helmet 1 12500.00
2 rows selected.
```

7. Customer Purchase History with Payment

Adds payment information to the customer's purchase history.

```
SELECT C.ID, C.NAME, O.ORDER_ID, P.PRODUCT_NAME,
O.TOTAL_AMOUNT, PY.PAYMENT_METHOD, PY.PAYMENT_STATUS
FROM Customer C
JOIN Orders O ON C.ID = O.CUSTOMER_ID
JOIN Order_Details OD ON O.ORDER_ID = OD.ORDER_ID
JOIN Product P ON OD.PRODUCT_ID = P.PRODUCT_ID
LEFT JOIN Payment PY ON O.ORDER_ID = PY.ORDER_ID
ORDER BY C.ID;
```

OUTPUT FORMAT (example)

```
SQL> SELECT C.ID, C.NAME, O.ORDER_ID, P.PRODUCT_NAME,
2 O.TOTAL_AMOUNT, PY.PAYMENT_METHOD, PY.PAYMENT_STATUS
3 FROM Customer C JOIN Orders O ON C.ID = O.CUSTOMER_ID
4 JOIN Order_Details OD ON O.ORDER_ID = OD.ORDER_ID
5 JOIN Product P ON OD.PRODUCT_ID = P.PRODUCT_ID
6 LEFT JOIN Payment PY ON O.ORDER_ID = PY.ORDER_ID
7 ORDER BY C.ID;
ID NAME ORDER_ID PRODUCT_NAME TOTAL_AMOUNT PAYMENT_METHOD STATUS
-- ----------- -------- --------------- ------------ -------------- ------
101 Customer A 5001 Duke Road Bike 190000.00 UPI PAID
102 Customer B 5002 Bike Helmet 12500.00 CARD PAID
2 rows selected.
```

8. Multi-Table Business Report

Generates a consolidated report of customers, orders, products, quantities, and payment status.

```
SELECT C.ID, C.NAME, COUNT(DISTINCT O.ORDER_ID) AS TOTAL_ORDERS,
SUM(OD.QUANTITY) AS ITEMS_PURCHASED,
SUM(O.TOTAL_AMOUNT) AS ORDER_VALUE
FROM Customer C
LEFT JOIN Orders O ON C.ID = O.CUSTOMER_ID
LEFT JOIN Order_Details OD ON O.ORDER_ID = OD.ORDER_ID
GROUP BY C.ID, C.NAME
ORDER BY C.ID;
```

OUTPUT FORMAT (example)

```
SQL> SELECT C.ID, C.NAME, COUNT(DISTINCT O.ORDER_ID) AS TOTAL_ORDERS,
2 SUM(OD.QUANTITY) AS ITEMS_PURCHASED,
3 SUM(O.TOTAL_AMOUNT) AS ORDER_VALUE
4 FROM Customer C LEFT JOIN Orders O ON C.ID = O.CUSTOMER_ID
5 LEFT JOIN Order_Details OD ON O.ORDER_ID = OD.ORDER_ID
6 GROUP BY C.ID, C.NAME ORDER BY C.ID;
ID NAME TOTAL_ORDERS ITEMS_PURCHASED ORDER_VALUE
-- ----------- ------------ --------------- -----------
101 Customer A 1 1 190000.00
102 Customer B 1 1 12500.00
2 rows selected.
```

9. Payment Transaction Report

Displays payment transaction details for each order.

```
SELECT O.ORDER_ID, O.CUSTOMER_ID, O.TOTAL_AMOUNT,
PY.PAYMENT_ID, PY.PAYMENT_DATE,
PY.PAYMENT_METHOD, PY.PAYMENT_STATUS
FROM Orders O
LEFT JOIN Payment PY
ON O.ORDER_ID = PY.ORDER_ID
ORDER BY O.ORDER_ID;
```

OUTPUT FORMAT (example)

```
SQL> SELECT O.ORDER_ID, O.CUSTOMER_ID, O.TOTAL_AMOUNT,
2 PY.PAYMENT_ID, PY.PAYMENT_DATE,
3 PY.PAYMENT_METHOD, PY.PAYMENT_STATUS
4 FROM Orders O LEFT JOIN Payment PY ON O.ORDER_ID = PY.ORDER_ID
5 ORDER BY O.ORDER_ID;
ORDER_ID CUSTOMER_ID TOTAL_AMOUNT PAYMENT_ID PAYMENT_DATE PAYMENT_METHOD STATUS
-------- ----------- ------------ ---------- ------------ -------------- ------
5001 101 190000.00 9001 21-SEP-26 UPI PAID
5002 102 12500.00 9002 21-SEP-26 CARD PAID
2 rows selected.
```

Conclusion

JOIN operations help connect related Duke Bike records and produce useful order, purchase-history, and payment reports. The black terminal-style panels illustrate the expected SQL*Plus presentation; sample output rows are examples,

not verified results from the user's database.